

Superposition Among the Computer Instruction Set Architectures

Lightweight fault isolation (LFI) is a highly versatile and effective means of performing software isolation (SFI). Process isolation is a critical assumption of modern compute systems: it is of utmost importance that separate programs have access to separate memory segments, and cannot access each other data/execution space without proper permissions. Hardware fault isolation (HFI), although highly effective, has limitations in that changing domains (such as context switching) can be expensive, even with microkernels and other optimizations. SFI has thus been of interest in recent years due its ability to change domains with higher performance, and because of the recent proliferation of cloud computing. Cloud computing may have to switch between thousands of short lived processes, and as such it is of critical importance that changing domains is not costly (which SFI and LFI in particular guarantees). This also has useful performance benefits for microkernels, where many inter-procedural calls can require frequent changing of domains, and LFI can deliver on performance while still providing process isolation.

Thus, the LFI system was developed, where the compiler toolchain is intercepted and before the assembly language is assembled and linked into an executable ELF file, load, stores, and jumps are identified, and masked via clearing the highest order 32 bits so as to artificially constrain the programs memory and execution range to within 4 GB. Without going into exorbitant details, the system was created for the x86, ARM, and RISC-V instruction sets. There are various optimizations that LFI deploys between each architecture, as a result of the specific implementation and conventions of various instruction types and the overall architectural design. As a result, some features of LFI (such as DeCL: deterministic client where the program deterministically terminates after a certain number of clock cycles) are much easier to implement in some architectures when compared to others. It would be a useful feature if one could program a certain feature of LFI for one architecture, compile it into an executable file, and then be able to "translate" the executable from one architecture to another. This would effectively port over optimization features present in the LFI system for one architecture to another.

In fact, it is generally useful if binaries compiled to one architecture can be naturally translated and executed on another architecture. One might state that a central purpose of compilers is their ability to take high level source languages and translate them for arbitrary architectures, and this is what grants the high level language portability and usability. However, some ISAs have particular features (such as fixed instruction length, flag setting on various instructions, amount of registers, complexity of loads and stores, etc) that are easier to deploy some program charac-

teristics when compared to others ISAs. Moreover, it is not uncommon for a binary to exist without the complementary source code being readily available, and this creates complications if the binary ELF is created for one architecture and cannot be executed on another. Thus, it is of interest to see if binaries can be generated where they may be executed on their intended ISA, and also be translated to another ISA if necessary for execution in another context.

In this particular project, we seek to superimpose the x86 and ARM instruction set architectures. More specifically, we wish to translate an ELF file designed for execution on x86 machines to an ELF file designed for execution of ARM machines. The naive approach is to take the executable for an x86 machine, and for each instruction, perform a 1-1 mapping of an x86 instruction to an ARM instruction. That is, a load issued in x86 has an equivalent load issued in ARM, a branch in x86 has an equivalent branch issued in ARM, etc. However, there are two major problems with this approach:

- **Instruction complexity:** x86 supports addressing modes that are significantly more complex when compared to the ARM equivalents for MOV, ADD, SUB, IMUL, and other arithmetic and memory motion instructions. For example, consider the following x86 instruction: `mov 4(%rdi, %rsi, 2), %rax`. There is no single instruction in the ARM ISA that allows for a load at an index offset scaled by 2, shifted by 4, and moves it into the return register. Instead, we would use the following two ARM instructions in order to encode the above memory operation:

```
1 add tmp, x20, x19, 1 lsl
2 ldr tmp, [tmp2, 4]
3 mov tmp, x0
```

Where we store an intermediate partial address in register tmp (a hypothetical register that in practice is set to register x24), from where the full pointer dereference occurs in register tmp2 (effectively register x25), and the resulting dereference may be moved into %rax. One may observe that the last MOV instruction is redundant (as will be discussed in the future research section), however this structure allows for portable substitution, in that the above scheme can also be used to substitute arithmetic instructions (such as ADD, SUB, etc)

- **Variable instruction lengths:** x86 instructions range from 1-15 bytes, whereas every ARM instruction is exactly 4 bytes. This problem is accentuated by the fact that not every instruction in x86 is represented by a single ARM instruction, which increases instruction length variability. The problem with this is that branch and jump targets will be misaligned if the binary ELF (and even individual instruction sequences) are substituted without proper padding, and therefore create a program with an entirely different semantic. Clearly, it is important that the translated binaries are equivalent in every way relevant to program execution, and thus this issue must be prevented.

Therefore, it is clear that x86 instructions must be matched with equivalent sequences of ARM instructions,

and that these equivalences are not only equivalent semantically (as in their result on the execution space is equivalent) but they are also equivalent in length. We thus propose the following two-phase process to correctly create a superposition of binaries, whilst :

1. **Format:** We wish to format the code so that every x86 instruction is prepared to be substituted with an ARM equivalent during the compilation process, so that the final ELF can eventually be superpositioned between the two architectures. In effect, this is accomplished by adding the correct number of NOP instructions so that every instruction is 4 byte aligned and there exists a correct sequence of ARM instructions. For example, although the x86 instruction `movl -96(%rbp,%rax,4), %eax` is only 4 bytes in length, it requires at least 2 ARM instructions (3 when deployed within this project) to achieve a proper substitution, so additional NOP commands are inserted to perform this.
2. **Translate:** We wish to obtain an ELF file that is designed for x86 execution, and loop over it to substitute every x86 instruction with their ARM equivalents. It is assumed that sufficient padding through NOPs has already been placed, so the program simply loops through the ELF file and substitutes every valid instruction it encounters.

The formatting phase is accomplished by first compiling high level code to an assembly file (if it is not already in x86 assembly), from which a go program loops through every line of code, and produces the padded assembly with the original x86 instructions and a certain number of NOP instructions. The number of NOP instructions is determined by the number of bytes needed to make the total byte sequence (instruction and NOPs) a multiple of 4, as well as the number of bytes needed for an equivalent ARM instruction. Regular expressions are used to parse memory movement operations, arithmetic operations, jumps, comparators, and more, and depending on the instruction parsed, a certain number of NOP operations are placed for padding. For the specific case of memory movement and arithmetic, sophisticated regular expressions are used to identify the specific type of memory access (as both MOV and arithmetic instructions can dereference a pointer or store to memory in x86) to determine both instruction length, as well as to determine if there is an index offset register associated with the memory operation. If there is, then since it requires 3 ARM instructions for adequate translation, the instruction will be padded to 12 bytes as opposed to 8. This is seen with the earlier example of `movl -96(%rbp,%rax,4), %eax`, which is only 4 bytes in length. However, since its ARM equivalent (for the purposes of this project) is

```
1 add tmp2, x5, x0, 2 lsl
2 ldr tmp, [tmp2, -96]
3 mov tmp, x0
```

Which is 12 bytes in length, thus 8 bytes are added in NOP instructions for the x86 equivalent (One might have noticed that the final destination is x0 as opposed to w0. A future optimization will correctly deal with registers smaller than 64 bits)

The translation phase is relatively simple for instructions that are not MOV or arithmetic operations: the ELF file is loaded into memory, the x86 instructions are decoded, and if a particular instruction type matches, such as jumps and branches, comparators, push and pop, etc, then the ARM equivalent is written to the same file so as to overwrite the x86 instructions. MOV and arithmetic instructions are more complicated, as they might have memory operations. If there is a load being performed by the instruction before the move or arithmetic, then it is detected via an x86 decoder, and a preamble of instructions are issued so as to retrieve the data from memory. For example, for the instruction `mov -96(%rbp,%rax,4), %rax`, the preamble issued is:

```
1 add tmp2, x5, x0, 2 lsl
2 ldr tmp, [tmp2, -96]
```

After the data is retrieved and stored in a temporary register (which is possible as ARM has twice as many registers as x86), the temporary is treated as if it were the original register from which the x86 instruction performed a MOV or arithmetic operation on, and the regular ARM equivalent instruction is issued for the temporary:

```
1 mov tmp, x0
```

However, an optimization is performed at this stage: an ARM instruction can be saved from the preamble if there is no index register and scale for the index. For example, for the instruction `mov 8(%rdi), %rsi`, the preamble is simply:

```
1 ldr tmp, [x19, 8]
```

with the primary instruction issued being:

```
1 mov tmp, x20
```

since ARM instructions can perform loads with constant offsets.

Stores are handled in a similar way, except an epilogue sequence of instructions is issued as opposed to a preamble to handle the storing operation after the arithmetic or move is performed. As an example, `mov %rax, -96(%rbp,%rax,4)` is translated to:

```
1 mov x0, tmp
2 add tmp2, x5, x0, 2 lsl
3 str tmp, [tmp2, -96]
```

Future Research: An obvious new direction is to translate binaries from an ARM architecture to an x86 architecture. Many of the same principles and techniques will still apply, however a new interesting complication is that there are twice as many ARM registers as there are x86 registers. Somehow, the greatly increased number of registers should be mapped onto a smaller register set, and there exist many potential ways in which this problem can be solved. One such way is to allocate a virtual page table in memory to house the additional registers. Another solution may be to potentially generate a graph of the register dependencies, and potentially identify redundant registers for deallocation and simplification. Nonetheless, translating binaries from ARM to x86 (and even to RISC-V and other architectures) is an important research direction to consider.

Another area for future research is to explore NOP minimization, as they amount to wasted clock cycles and increase the size of the binary considerably. NOP instruction placement is unavoidable due to differences in instruction lengths between the architectures, and this project already attempts to minimize such via placing supplementary NOPS when complex memory operands are detected for x86 instructions. Nonetheless, the ARM instruction set is rich with various commands that can be used to replace various x86 commands more efficiently than direct substitution, and perhaps bundles of x86 instructions can be mathed and replaced with equivalent bundles of ARM instructions for added performance.

A final interesting area of research is to explore methods of translating complex instructions, such as vector instructions and multithreading instructions, from one binary to another. Presumably, translating vector instructions from one binary to another is not terribly complicated: even if the number of bytes housed by each vector differ, additional sequences of vector instructions can be placed to accurately simulate the correct number of bytes held by a vector from one binary to the next. What is more complicated is multithreading, as x86 and ARM have different threading models